Compiladores 2025/26

Análise Sintática Descendente

Raul Barbosa

Departamento de Engenharia Informática
Faculdade de Ciências e Tecnologia
Universidade de Coimbra

· U C ·

Análise sintática descendente

Árvore construída a partir da raiz com derivação pela esquerda.

O analisador descendente pode "prever" qual a produção correta se um símbolo de *lookahead* indicar univocamente a escolha correta.

- Muitas gramáticas úteis têm esta propriedade.
- A propriedade, em si, deve ser especificada com rigor.
- Nem sempre é trivial aplicar o algoritmo:
  - Quando o primeiro símbolo do lado direito é um não-terminal
  - Quando se *anula* um não-terminal com uma produção $A \rightarrow \epsilon$

Conjuntos FIRST e FOLLOW

FIRST(α)

A análise sintática preditiva requer informação sobre os primeiros símbolos que podem ser gerados por uma produção.

- α é uma sequência de símbolos gramaticais (terminais ou não).
- FIRST(α) é o conjunto de terminais que são os primeiros símbolos em uma ou mais sequências geradas por α.
- $\epsilon \in$ FIRST(α) se α puder derivar ϵ, isto é, se α for anulável[1].

FOLLOW(A)

É o conjunto de terminais que se podem seguir a A numa derivação, sendo A um não-terminal.

[1] Alguns autores incluem ϵ nos conjuntos FIRST, enquanto outros representam à parte o conjunto de não-terminais anuláveis NULLABLE.

Cálculo dos conjuntos FIRST

Calcular FIRST(X) para todos os símbolos X da gramática:

1. Se X é terminal, então FIRST(X) $= \{X\}$
2. Se X é não-terminal e $X \rightarrow A_1 A_2 \cdots A_k$ for uma produção, então incluir t em FIRST(X) se $t \in$ FIRST(A_i) para algum i e se $\epsilon \in$ FIRST(A_j) para todo o $j < i$
   - Ou seja, FIRST(A_1) pertence definitivamente a FIRST(X)
   - Se $\epsilon \notin$ FIRST(A_1), então não há mais nada a acrescentar
   - Se $\epsilon \in$ FIRST(A_1), então FIRST(A_2) também pertence a FIRST(X) e assim sucessivamente
3. Se $X \rightarrow \epsilon$ for uma produção, acrescentar ϵ a FIRST(X)
4. Repetir até que mais nenhum conjunto FIRST se altere

Cálculo dos conjuntos FIRST para sequências $X_1 X_2 \cdots X_n$

Calcular $\text{FIRST}(X_1 X_2 \cdots X_n)$ para sequências arbitrárias:

- Colocar $\text{FIRST}(X_1) \setminus \epsilon$ no conjunto $\text{FIRST}(X_1 X_2 \cdots X_n)$
- Se $\epsilon \in \text{FIRST}(X_1)$, acrescentar $\text{FIRST}(X_2) \setminus \epsilon$
- Se $\epsilon \in \text{FIRST}(X_1) \wedge \epsilon \in \text{FIRST}(X_2)$, acrescentar $\text{FIRST}(X_3) \setminus \epsilon$
- Se, para todo o k, $\epsilon \in \text{FIRST}(X_k)$, acrescentar ϵ

Exemplo: gramática de expressões aritméticas

$$
\begin{array}{lcl}
E & \rightarrow & T\ E' \\
E' & \rightarrow & +\ T\ E' \\
 & | & -\ T\ E' \\
 & | & \epsilon \\
T & \rightarrow & F\ T' \\
T' & \rightarrow & *\ F\ T' \\
 & | & /\ F\ T' \\
 & | & \epsilon \\
F & \rightarrow & id \\
 & | & (\ E\ )
\end{array}
$$

$$\text{FIRST}(F) = \text{FIRST}(id) \cup \text{FIRST}(\,(E)\,) = \{id, (\}$$
$$\text{FIRST}(T) = \text{FIRST}(FT') = \text{FIRST}(F) = \{id, (\}$$
$$\text{FIRST}(E) = \text{FIRST}(TE') = \text{FIRST}(T) = \{id, (\}$$
$$\text{FIRST}(E') = \text{FIRST}(+TE') \cup \text{FIRST}(-TE') \cup \text{FIRST}(\epsilon) = \{+, -, \epsilon\}$$
$$\text{FIRST}(T') = \text{FIRST}(*FT') \cup \text{FIRST}(/FT') \cup \text{FIRST}(\epsilon) = \{*, /, \epsilon\}$$

Cálculo dos conjuntos FOLLOW

Calcular FOLLOW(A) para todos os não-terminais A da gramática:

1. Colocar \$ no FOLLOW(S), com S o símbolo inicial e \$ o EOF
2. Para cada produção $Y \rightarrow \alpha A \beta$
   acrescentar FIRST(β) $\setminus \epsilon$ ao FOLLOW(A)
3. Para cada produção $Y \rightarrow \alpha A \beta$ onde $\epsilon \in$ FIRST(β)
   acrescentar FOLLOW(Y) ao FOLLOW(A)
4. Para cada produção $Y \rightarrow \alpha A$
   acrescentar FOLLOW(Y) ao FOLLOW(A)
5. Repetir até que mais nenhum conjunto FOLLOW se altere

Exemplo: gramática de expressões aritméticas

$$
\begin{array}{lcl}
E & \rightarrow & T\ E' \\
E' & \rightarrow & +\ T\ E' \\
 & | & -\ T\ E' \\
 & | & \epsilon \\
T & \rightarrow & F\ T' \\
T' & \rightarrow & *\ F\ T' \\
 & | & /\ F\ T' \\
 & | & \epsilon \\
F & \rightarrow & id \\
 & | & (\ E\ )
\end{array}
$$

$$\text{FOLLOW}(E) = \{\$\} \cup \text{FIRST}(\,)\,) = \{\$,)\}$$
$$\text{FOLLOW}(E') = \text{FOLLOW}(E) = \{\$,)\}$$
$$\text{FOLLOW}(T) = \text{FIRST}(E') \setminus \epsilon \cup \text{FOLLOW}(E) \cup \text{FOLLOW}(E') = \\ = \{+, -, \$,)\}$$
$$\text{FOLLOW}(T') = \text{FOLLOW}(T) = \{+, -, \$,)\}$$
$$\text{FOLLOW}(F) = \text{FIRST}(T') \setminus \epsilon \cup \text{FOLLOW}(T) \cup \text{FOLLOW}(T') = \\ = \{*, /, +, -, \$,)\}$$

Gramáticas LL(1)

Pode-se construir analisadores preditivos para gramáticas LL(1)

- Um analisador preditivo é um analisador descendente recursivo que não requer tentativa e erro.
- Analisadores preditivos são adequados para uma classe de gramáticas chamada LL(1).
- **L**eft-to-right input scan, producing a **L**eftmost derivation, using **1** lookahead token.
- Existe a classe LL(k) mas na prática $k = 1$ já que qualquer gramática LL(k) pode ser trivialmente convertida em LL(1).

Construção da tabela do analisador LL(1)

Para um dado não-terminal A e o próximo símbolo lido t queremos saber em que condições usamos a produção $A \to \alpha$

Temos que $T[A, t] = \alpha$ em dois casos:

1. $\alpha \to \cdots \to t\beta$
   - α pode derivar (em 0 ou mais passos) t na primeira posição.
   - Portanto, $t \in \text{FIRST}(\alpha)$.
2. $\alpha \to \cdots \to \epsilon$ e $S \to \cdots \to \beta A t \gamma$ mas $\alpha \not\to t$
   - A não pode derivar t mas pode desaparecer (anular-se).
   - t pode aparecer imediatamente a seguir a A noutra produção.
   - Portanto, $t \in \text{FOLLOW}(A)$.

Algoritmo para construção da tabela do analisador LL(1)

Para cada produção $A \rightarrow \alpha$ da gramática:

1. Para cada terminal $t \in \text{FIRST}(\alpha)$, colocar $A \rightarrow \alpha$ em $T[A, t]$
2. Se $\epsilon \in \text{FIRST}(\alpha)$, α é anulável, então para cada terminal $t \in \text{FOLLOW}(A)$, incluir $A \rightarrow \alpha$ em $T[A, t]$
   - Se $\$ \in \text{FOLLOW}(A)$, incluir igualmente $A \rightarrow \alpha$ em $T[A, \$]$

- Se, no final, não existir qualquer produção numa dada célula $T[A, b]$ então $T[A, b] = \text{error}$.
- Por definição, uma gramática é LL(1) se e só se a respetiva tabela LL(1) não contiver mais de uma entrada em cada célula.

Exemplo: tabela do analisador LL(1)

$$
\begin{array}{lll}
(1) & E \rightarrow & T\ E' \\
(2) & E' \rightarrow & +\ T\ E' \\
(3) & \quad\quad | & -\ T\ E' \\
(4) & \quad\quad | & \epsilon \\
(5) & T \rightarrow & F\ T' \\
(6) & T' \rightarrow & *\ F\ T' \\
(7) & \quad\quad | & /\ F\ T' \\
(8) & \quad\quad | & \epsilon \\
(9) & F \rightarrow & id \\
(10) & \quad\quad | & (\ E\ )
\end{array}
$$

	$	(	id	/	*	-	+	)
E		(1)	(1)					
E'	(4)					(3)	(2)	(4)
T		(5)	(5)					
T'	(8)			(7)	(6)	(8)	(8)	(8)
F		(10)	(9)					

Exemplo de gramática que não é LL(1)

$$
\begin{array}{lrcl}
(1) & F & \rightarrow & id \\
(2) & & | & id \ (\ Args \ ) \\
(3) & Args & \rightarrow & F \ Arg \\
(4) & Arg & \rightarrow & , \ F \ Arg \\
(5) & & | & \epsilon
\end{array}
$$

	$	id	(	,	)
F		(1) (2)			
Args		(3)			
Arg				(4)	(5)

Tabela do analisador LL(1)

O conflito impede a escolha determinística da produção a aplicar, com base no símbolo de *lookahead* e no não-terminal atuais, logo a gramática não é LL(1).

Exercício

Gramática de chamadas de funções, fatorizada à esquerda:

$$
\begin{array}{lcl}
F & \rightarrow & id \;\; OptArgs \\
OptArgs & \rightarrow & (\;\; Args \;\;) \\
 & | & \epsilon \\
Args & \rightarrow & F \;\; Arg \\
Arg & \rightarrow & , \;\; F \;\; Arg \\
 & | & \epsilon
\end{array}
$$

Algumas frases geradas por esta gramática:

id id(id) id(id,id) id(id(id),id,id)

Mostra-se que esta gramática é LL(1) apresentando a tabela do respetivo analisador sintático.

Resolução

$$
\begin{array}{lll}
(1) & F \rightarrow id\ OptArgs & \\
(2) & OptArgs \rightarrow (\ Args \ ) & \\
(3) & \qquad\qquad\quad | \ \epsilon & \\
(4) & Args \rightarrow F\ Arg & \\
(5) & Arg \rightarrow \ ,\ F\ Arg & \\
(6) & \qquad\quad | \ \epsilon &
\end{array}
$$

	FIRST	FOLLOW
F	id	$,)
OptArgs	(ϵ	$,)
Args	id	)
Arg	, ϵ	)

	$	id	(	,	)
F		(1)			
OptArgs	(3)		(2)	(3)	(3)
Args		(4)			
Arg				(5)	(6)

Tabela do analisador sintático LL(1)

Compiladores 2025/26

Análise Sintática Descendente

Raul Barbosa

Departamento de Engenharia Informática
Faculdade de Ciências e Tecnologia
Universidade de Coimbra

· U C ·